Blockchain and Decentralization

Lecture 1 — An Introduction

Pierre Sutra

Télécom SudParis

CSC5006

version 1.0

Organization

- 11 lectures (1.5h), each followed by a 1.5h session (plus 1–2h at home): 5 **practicals** and 6 sets of **exercises**
  - exercises on paper (*not graded*)
  - practicals in groups of 2–3 students: 40% of the grade (top 3 only!)
  - final exam: 60% of the grade
- every lecture comes with readings that support and expand it
- this course is about **technology**, not about cryptocurrency speculation
  - what the systems guarantee, under which assumptions, and at which cost
  - nothing here is investment advice

$$\textit{grade} = 0.4 \times \mathrm{mean}(\text{3 best practicals}) + 0.6 \times \textit{exam}$$

Practicals

1. practicals are done in **groups of 2–3 students**
2. each group uploads on Moodle an archive (zip, tgz) with
   - a short **report** of around 4–5 pages
   - the **code**
3. the **average of the 3 best practicals** counts for the final grade

- deadline for all practicals: **January 31st, 2027**
- it is best to **upload regularly** rather than wait until the end

The use of GenAI (e.g., Gemini, Claude) in this course is **strictly limited** to the following supportive use cases:

- helping you understand key computing concepts and notions
- fixing typos and grammar mistakes in your writing
- reviewing or debugging code or configurations that you wrote yourself

You **may not** use GenAI to do the core work of the course. In particular, to:

- generate the logic, code, or configuration files of your practicals (even partially)
- find out answers to the questions asked in the practicals

Disclosure if you use GenAI for any of the permitted use cases, briefly disclose how you used it in your report (at the very beginning)

Responsibility you take full responsibility for your work: GenAI makes mistakes, and you must be able to understand where it is wrong

Course roadmap

Lectures 1–2 building blocks: cryptography, data dissemination

Lectures 3–5 agreement: consensus (FLP, Paxos), replicated state machines, Byzantine failures (PBFT)

Lectures 6–8 blockchains proper: distributed ledgers (Proof-of-X, layer 2), wallets, smart contracts and privacy, DeFi

Lectures 9–11 advanced topics: decentralized storage and compute, CRDTs, cross-chain transactions

Agenda (1/2)

#	date	lecture	1.5h session
1	27 Oct	Introduction	**practical 1**
2	3 Nov	Data dissemination and mempool	**practical 2***
3	10 Nov	Consensus, FLP, and Paxos	exercises
4	24 Nov	Byzantine failures	exercises
5	1 Dec	Replicated state machines	**practical 3***
6	15 Dec	Distributed ledgers	exercises
7	5 Jan 2027	Wallets, smart contracts, and privacy	**practical 4**
8	8 Jan	DeFi and cryptocurrencies	exercises
9	12 Jan	Decentralized storage and compute	exercises
10	18 Jan	CRDTs	**practical 5***
11	19 Jan	Cross-chain transactions	exercises
	26 Jan	**final exam**	

($*$) teaching assistant: T. Nguyen

Today

A general picture of the history and state of blockchain technologies.

- why decentralize?
- two lineages: distributed computing and safe computing; today, safe computing
- Bitcoin, the first blockchain

Most notions are only brushed here and developed later.

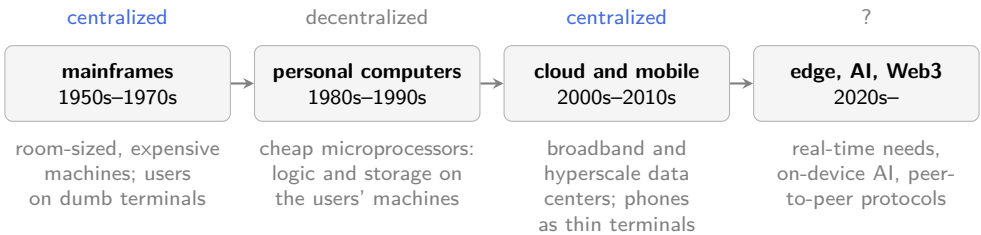
Part	Time
Context	5’
Blockchain 101	7’
Safe computing	18’
Bitcoin	10’
Wrap-up	2’

1. Context: why decentralize?
2. Blockchain 101
3. Safe computing
4. Bitcoin: the first blockchain
5. Wrap-up

"Who decides whether your payment goes through?
Who can edit the record of who owns what?"

- today, the answer is almost always: **one company or one institution**
  - a bank, a cloud provider, a platform

A back-and-forth between centralization and decentralization



- the pendulum follows the **cost of compute and bandwidth**: expensive machines are shared, cheap ones are owned, and cheap bandwidth lets us rent them again
- today, the technology swings back to the edge; yet the **control** of platforms and data stays with a few companies (next slide)

We live in a world of unprecedented centralization

- hyperscalers (Amazon, Microsoft, Google, and Meta) hold **more than half** of the total computing power in the West
- in China, the situation is not much different

Can we decentralize (again) computing infrastructures?

- **single point of failure:** a region of a cloud provider goes down, and half of the Web with it

What is wrong with centralization?

- **single point of failure:** a region of a cloud provider goes down, and half of the Web with it
- **trust:** the operator can censor, reverse, or rewrite; users must trust that it will not

What is wrong with centralization?

- **single point of failure:** a region of a cloud provider goes down, and half of the Web with it
- **trust:** the operator can censor, reverse, or rewrite; users must trust that it will not
- **lock-in and rent extraction:** data and users are captive

What is wrong with centralization?

- **single point of failure:** a region of a cloud provider goes down, and half of the Web with it
- **trust:** the operator can censor, reverse, or rewrite; users must trust that it will not
- **lock-in and rent extraction:** data and users are captive
- **attack surface:** one breach exposes everybody

What is wrong with centralization?

- **single point of failure:** a region of a cloud provider goes down, and half of the Web with it
- **trust:** the operator can censor, reverse, or rewrite; users must trust that it will not
- **lock-in and rent extraction:** data and users are captive
- **attack surface:** one breach exposes everybody

But decentralization is not free: coordination is hard, performance lower, and in an open system there might be no trust and a possibly unknown set of participants.

→ *This course is about how to tackle all these problems.*

Can we decentralize computing infrastructures *safely*?

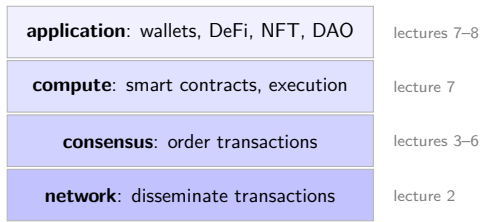
This course answers positively, by presenting key notions that provide the building blocks of decentralized and safe computing infrastructures:

- distributed protocols
- cryptographic constructs

1. Context: why decentralize?
2. Blockchain 101
3. Safe computing
4. Bitcoin: the first blockchain
5. Wrap-up

A blockchain is a **replicated storage**, maintained by parties that do not trust one another, and **robust to tampering**.

- a "computer that nobody owns"
- the basis of **Web3**: decentralized applications (dApps), where users own their identity and assets with keys
- anyone can submit transactions to access the storage
- the storage is a list, the *ledger*, of *blocks* of transactions



What are blockchains used for?

Usage	What it does	Examples
payments	move value without a bank	Bitcoin, stablecoins (USDT, DAI)
smart contracts	programs that every node executes	Ethereum, Solana
tokens and NFTs	ownership of digital assets; an NFT (*non-fungible token*) is unique	ERC-20, ERC-721

What are blockchains used for?

Usage	What it does	Examples
payments	move value without a bank	Bitcoin, stablecoins (USDT, DAI)
smart contracts	programs that every node executes	Ethereum, Solana
tokens and NFTs	ownership of digital assets; an NFT (*non-fungible token*) is unique	ERC-20, ERC-721
DAOs	*decentralized autonomous organizations*, run by code and token votes	AssangeDAO, ApeCoin DAO

What are blockchains used for?

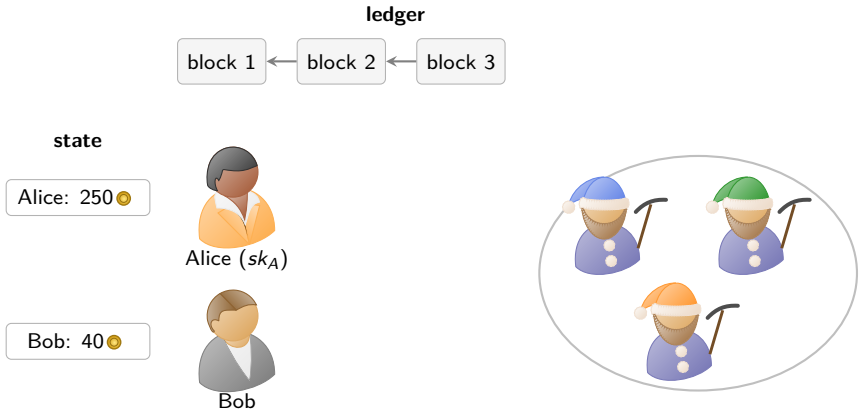
Usage	What it does	Examples
payments	move value without a bank	Bitcoin, stablecoins (USDT, DAI)
smart contracts	programs that every node executes	Ethereum, Solana
tokens and NFTs	ownership of digital assets; an NFT (*non-fungible token*) is unique	ERC-20, ERC-721
DAOs	*decentralized autonomous organizations*, run by code and token votes	AssangeDAO, ApeCoin DAO
decentralized finance	exchanges, lending, without intermediaries	Uniswap, Morpho
storage and compute	rent resources from strangers	Filecoin, Internet Computer
permissioned ledgers	consortia: supply chains, finance	Hyperledger Fabric

What are blockchains used for?

Usage	What it does	Examples
payments	move value without a bank	Bitcoin, stablecoins (USDT, DAI)
smart contracts	programs that every node executes	Ethereum, Solana
tokens and NFTs	ownership of digital assets; an NFT (*non-fungible token*) is unique	ERC-20, ERC-721
DAOs	*decentralized autonomous organizations*, run by code and token votes	AssangeDAO, ApeCoin DAO
decentralized finance	exchanges, lending, without intermediaries	Uniswap, Morpho
storage and compute	rent resources from strangers	Filecoin, Internet Computer
permissioned ledgers	consortia: supply chains, finance	Hyperledger Fabric

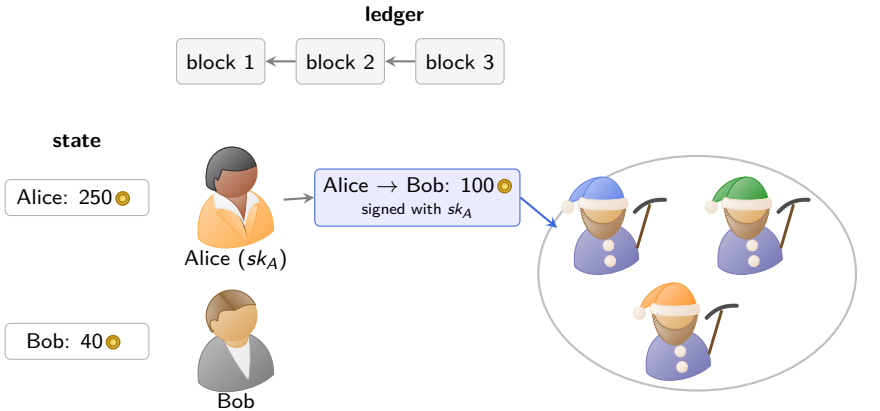
The **killer app** so far: decentralized finance, and stablecoins for payments.

A payment on a blockchain



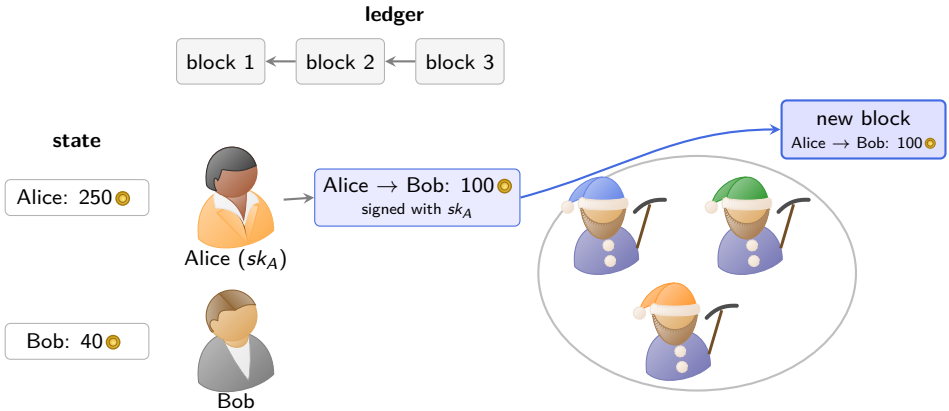
miners: the nodes that build the blocks, and are rewarded for it

A payment on a blockchain



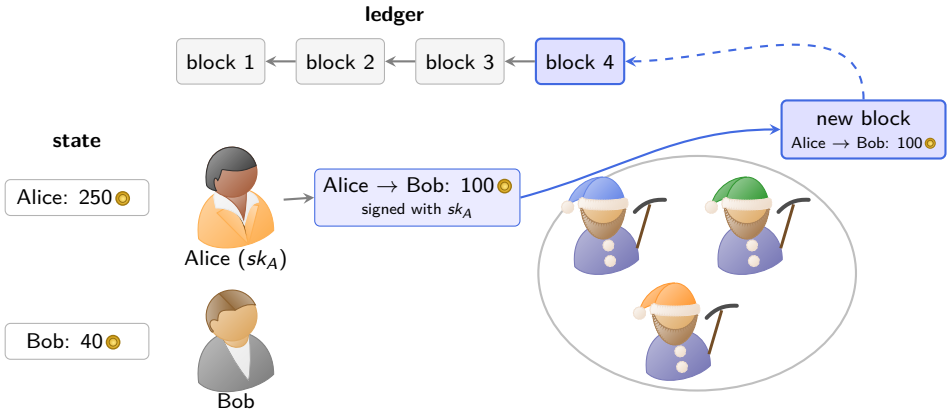
Alice signs a transaction that pays 100 to Bob, and sends it to the network.

A payment on a blockchain



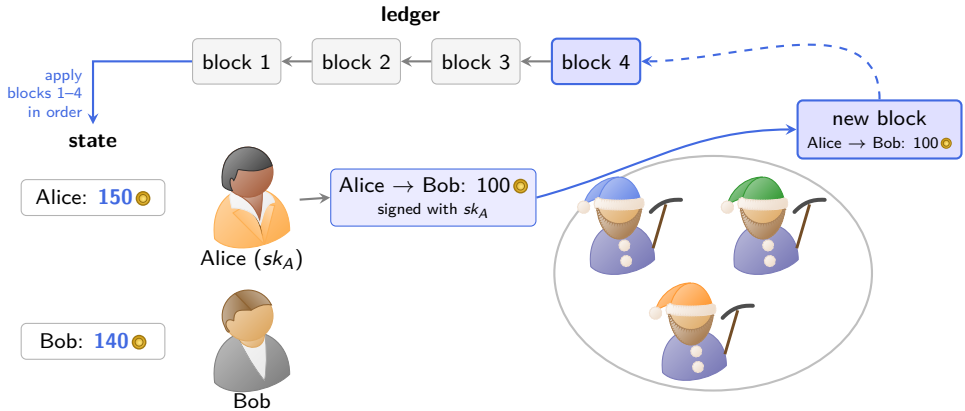
A miner puts the transaction in a newly minted block.

A payment on a blockchain



The block is appended to the **ledger**, replicated at every node.

A payment on a blockchain



Applying the ledger in order gives the **state**: the balances change.

Key notions

Transaction an elementary operation (e.g., read or write an account, transfer an asset); grouped in **blocks**

Ledger a shared data structure: a linear record of the transaction blocks; append-only; verifiable backwards

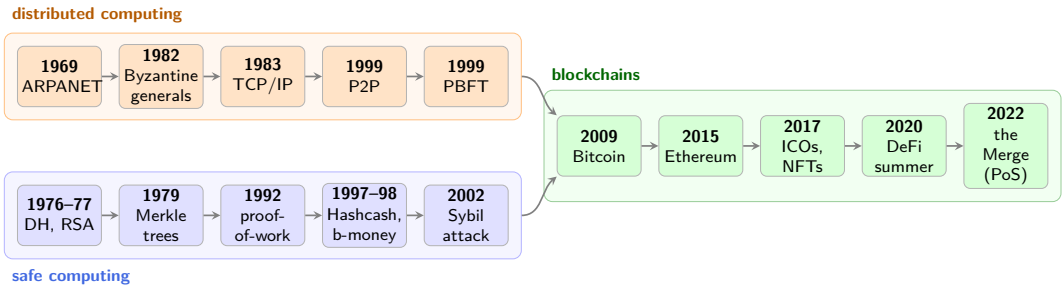
Mempool the protocol layer that disseminates pending transactions (lecture 2, practical 2)

Web3 decentralized applications (*dApps*) on blockchains; users hold their identity and assets with keys (lecture 7); also a marketing label

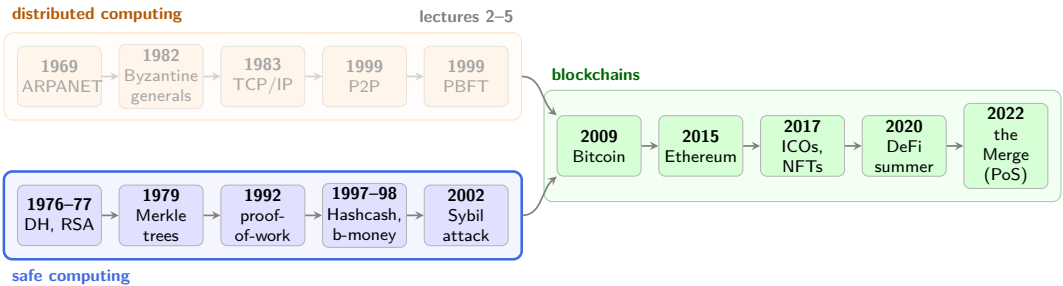
System model
- *permissioned*: participants are identified and well-known
- *permissionless*: anyone can join (Bitcoin, Ethereum); requires protection against Sybil attacks

Adversary crash vs. arbitrary faults; what fraction of participants (or of a resource) may be faulty

Chronology: two lineages that merge in blockchain



Chronology: two lineages that merge in blockchain

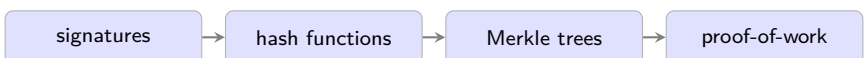


Distributed computing is the subject of lectures 2–5. The next section covers **safe computing**: the cryptographic tools that Bitcoin assembled.

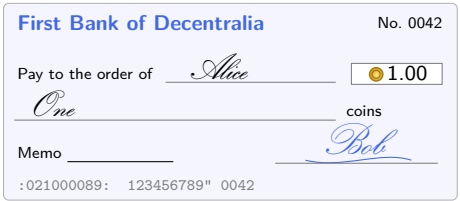
1. Context: why decentralize?
2. Blockchain 101
3. Safe computing
4. Bitcoin: the first blockchain
5. Wrap-up

How to make a system of distrusting machines safe?

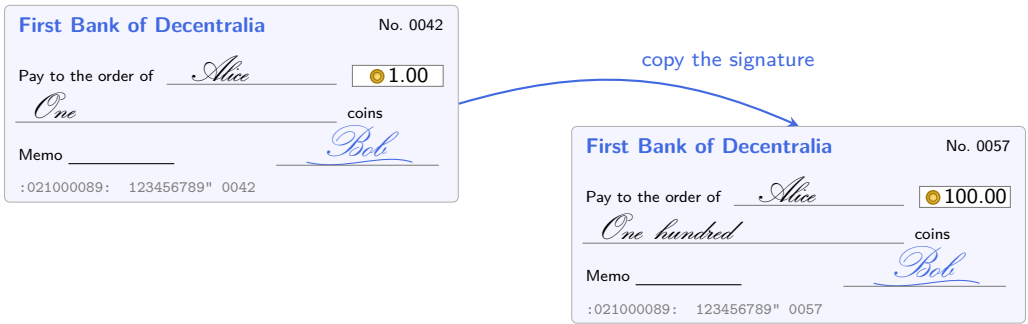
- we need
  1. **cryptographic tools** to authenticate, commit, and prove
  2. a way to make **identities costly**, so that an adversary cannot fake a crowd



- a physical signature **binds a transaction to its author**

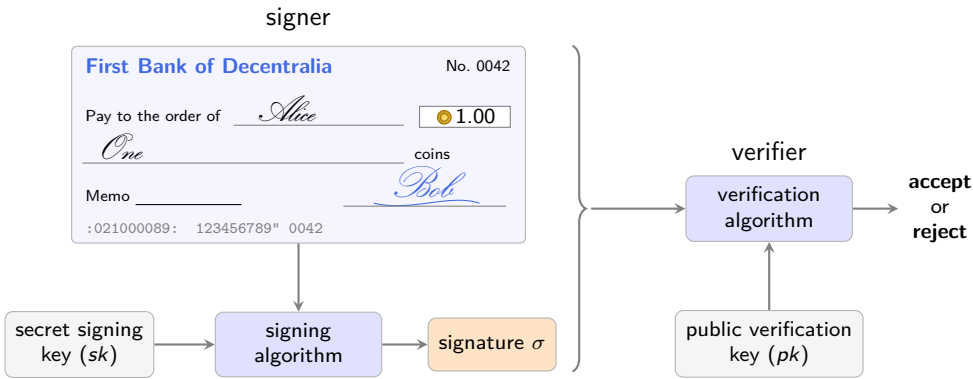


- a physical signature **binds a transaction to its author**



Problem in the digital world: anyone can copy Bob's signature from one document to another.

- solution: make the signature **depend on the document**



- *public-key* cryptography: *sk* stays secret, *pk* is published (first scheme: Rivest–Shamir–Adleman, RSA, 1977)

A signature scheme is a triple of algorithms:

- $Gen()$ outputs a key pair (pk, sk)
- $Sign(sk, m)$ outputs a signature σ
- $Verify(pk, m, \sigma)$ outputs *accept* or *reject*

Secure (informally): an adversary who sees signatures on many messages of its choice cannot forge a signature on a new message.

- **identity = public key**: no registry is needed to create an identity
- uses: identify users and machines, authorize transactions in blockchain

Families of signature schemes

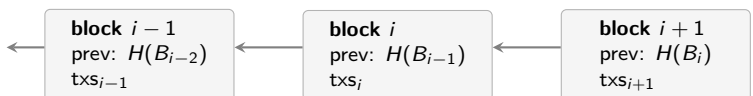
- **RSA** (Rivest–Shamir–Adleman): long signatures and keys; not used in blockchains
- **ECDSA** (Elliptic Curve Digital Signature Algorithm), **Schnorr**: short signatures (64 bytes): Bitcoin, Ethereum, Solana
- **BLS** (Boneh–Lynn–Shacham): **threshold** signatures: any t of n signers produce one short signature that proves a quorum signed; also aggregation (Ethereum consensus)
- **post-quantum**: long signatures (kilobytes)

$H : \{0,1\}^* \to \{0,1\}^{256}$ (SHA-256 in Bitcoin, Keccak-256 in Ethereum)

- deterministic, fast to compute, fixed-size output
- **preimage resistance**: given y, hard to find x such that $H(x) = y$
- **second-preimage resistance**: given x, hard to find $x' \neq x$ with $H(x') = H(x)$
- **collision resistance**: hard to find any $x \neq x'$ with $H(x) = H(x')$ (birthday bound: about 2^{128} work)
- avalanche effect: flipping one input bit flips about half of the output bits
- use: identify transactions and blocks

Hash pointers and the hash chain

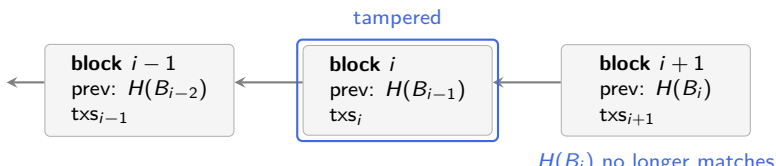
- a block stores a *hash pointer*: the hash of the content of the previous block



Warm-up: what does a collision-resistant hash buy you that a checksum (CRC) does not?

Hash pointers and the hash chain

- a block stores a *hash pointer*: the hash of the content of the previous block

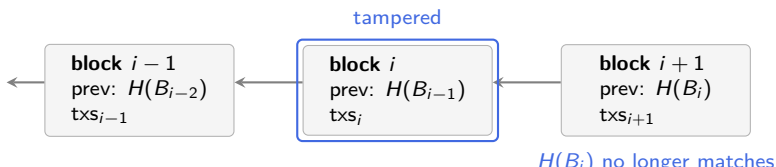


Warm-up: what does a collision-resistant hash buy you that a checksum (CRC) does not?

- changing an old block changes its hash, which breaks every later pointer
- the chain is **tamper-evident**: the origin of the term "blockchain"

Hash pointers and the hash chain

- a block stores a *hash pointer*: the hash of the content of the previous block



Warm-up: what does a collision-resistant hash buy you that a checksum (CRC) does not?

- changing an old block changes its hash, which breaks every later pointer
- the chain is **tamper-evident**: the origin of the term "blockchain"
- use: fix the order of the blocks, hence of the transactions

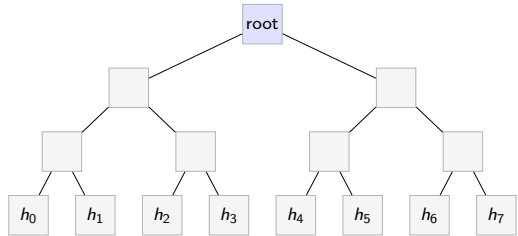
- a machine holds n transactions (or files)
- it wants to
  1. **commit** to the whole list with a short value
  2. later **prove** that one item belongs to the list, without sending the whole list
- hashing the concatenation of the list gives (1), but not (2)

$$\Rightarrow \text{ a } \textit{tree} \text{ of hashes } ^{\dagger}$$

[†] R. C. Merkle. "A Certified Digital Signature". *CRYPTO*. 1989.

Merkle trees: construction

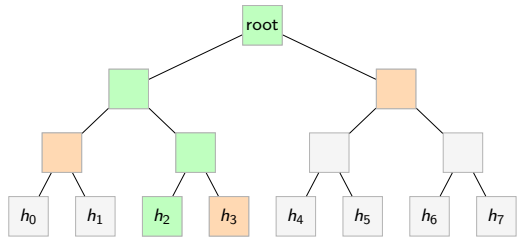
- leaves: $h_i = H(tx_i)$
- internal nodes: $H(left \,\|\, right)$
- the **root** is the commitment: 32 bytes, for any n
- odd number of nodes at a level? A rule must say what to do (Bitcoin duplicates the last node); the rule is part of the protocol



Merkle trees: membership proof

- proof for leaf i: the **siblings** along the path to the root: $\log_2 n$ hashes
- $n = 10^6$: about 20 hashes = 640 bytes
- the verifier recomputes the path bottom-up and compares with the published root
- it needs the leaf position, to know on which side to concatenate

path siblings (the proof)



Merkle trees: why is it secure?

- if a verifier accepts a proof for a value that is not in the list, then one has found a **collision** (or a second preimage) of H
- subtlety: leaves and internal nodes must be **domain-separated**
  - e.g., prefix `0x00` for leaves and `0x01` for internal nodes
  - otherwise, an internal node can be passed off as a leaf: a second-preimage attack on the tree
- you will try to break your own implementation in the practical

- the block header stores the Merkle root of the block's transactions: a small header (80 bytes in Bitcoin) still *commits* to all of them
- **light clients** (e.g., on a phone) download only headers, and ask a full node for a Merkle proof that a transaction is in a block
- comparing two chains or two states is cheap: compare roots, then descend where they differ
- **Bitcoin**: the root of the transactions in each block header; **Ethereum**: Merkle-Patricia tries for the state, the transactions, and the receipts
- beyond blockchains: Git, IPFS, anti-entropy in Apache Cassandra and Amazon Dynamo

- practical 1: **Merkle trees** — proving one thing without showing everything
- build the tree, produce and verify a proof, and try to break it
- in Go: `practicals/merkle`

Proof-of-work: motivation

- Dwork and Naor, *Pricing via Processing or Combatting Junk Mail*, CRYPTO '92[†]
- motivation: fight e-mail spam; sending a mail costs nothing
- idea: make each sender pay a **small computational cost** per message: negligible for a legitimate user, prohibitive for a spammer
- the recipient checks a *proof* that the work was done

> *. . . electronic junk mail presents a much greater problem. In particular, the simplicity of sending and low cost of sending electronic mail, and in particular the ease of sending the same message to many parties, all but invite abuse.* ***In this paper we suggest a computational approach to combatting the proliferation of electronic mail.***

[†] C. Dwork et al. "Pricing via Processing or Combatting Junk Mail". *CRYPTO*. 1992.

Proof-of-work: pricing functions

Computation has a cost, defined by a *pricing function* f. One can easily check that the cost was paid.

- f is "moderately easy" to compute
- f is not amenable to amortization: given $x_1, \ldots, x_m$, the amortized cost of computing $f(x_1), \ldots, f(x_m)$ is comparable to computing each $f(x_i)$
- given x and y, it is **easy** to determine whether $y = f(x)$

an *asymmetry*: hard to produce, easy to verify

Let H be a hash function. The following is a pricing function:[†]

given x, find y such that $H(x \,\|\, y)$ ends with δ zeros

```
H(x‖0) = 7f3a...e9
H(x‖1) = c21d...b4
H(x‖2) = 0e88...5a
...
H(x‖41873) = 5b0e...100   ← found
```

$\delta = 2$

- brute force: about 2^δ attempts
- verification: **one** hash
- δ sets the difficulty: $+1$ zero doubles the work
- in the anti-spam use, $x =$ recipient and date

[†] A. Back. *Hashcash — A Denial of Service Counter-Measure*. Technical report. 2002.

Why work is a useful resource: Sybil attacks

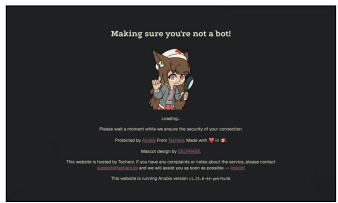
- in an open system, **identities are free**: an adversary creates many fake ones and wins any "one identity, one vote" protocol
- this is the *Sybil attack* (Douceur, 2002)[†]
- computing power is scarce and cannot be faked

tie influence to a **scarce resource** (CPU; later, stake)

[†] J. R. Douceur. "The Sybil Attack". *IPTPS*. 2002.

Work as a price, today: AI crawlers

- AI scrapers **overload small sites** and ignore `robots.txt`; blocking by IP address or user agent fails (both are free)
- **Anubis**: open source (2025), born when Amazon's crawler overloaded a Git server; inspired by Hashcash
  - the server asks the browser for a *nonce* such that $\mathrm{SHA\text{-}256}(\mathit{challenge} \,\|\, \mathit{nonce})$ starts with zeros
  - five zeros (default): seconds; six: minutes; a signed cookie then proves the work for a while
- **negligible** for a person reading a few pages, **large** for a crawler fetching millions



the proof-of-work challenge page of Anubis

Used by GNOME, FFmpeg, Wikimedia, UNESCO, ...

Limits cheap for a determined scraper; drains old devices; needs JavaScript

Summary: the building blocks

Tool	Provides	Used for
signature	authenticity, authorization	who may spend a coin
hash function	commitment, fingerprint	blocks, addresses, PoW
hash chain	tamper-evidence of history	the ledger
Merkle tree	short commitment, short proof of membership	block header, light clients
proof-of-work	costly identity, Sybil resistance	who may append a block

1. Context: why decentralize?
2. Blockchain 101
3. Safe computing
4. Bitcoin: the first blockchain
5. Wrap-up

Goal: mint and transfer e-money without a central authority (Wei Dai, 1998).[†]

- context: crypto-anarchism, the libertarians of the 90's
- each process has a public/private key pair (PKI); identity = public key
- each process replicates a database of who holds what
- to **mint**: solve a hard puzzle (PoW) and broadcast the solution
- to **transfer**: sign the transfer and broadcast it; upon reception, verify it and update the database

[†] W. Dai. *b-money*. Cypherpunks mailing list. 1998.

- how do processes **agree on the order** of transfers?
- what if a holder signs two conflicting transfers of the same coin to two recipients (*double spending*)?
- b-money relied on an under-specified agreement among participants, and was never implemented

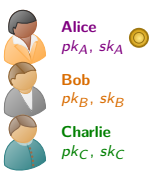
$\Rightarrow$ Bitcoin fills this gap.

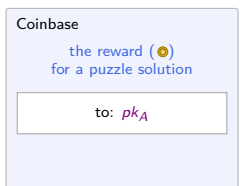


- published on a cryptography mailing list on 31 October 2008 by Satoshi Nakamoto; genesis block on 3 January 2009[†]
- timing: the 2008 financial crisis, and distrust towards bailed-out banks
- goal: electronic cash **without a trusted third party**, by combining the tools of the previous section

[†] S. Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. Cryptography mailing list. 2008.

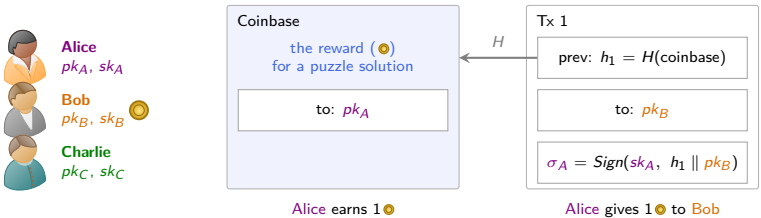
- a coin is **minted** by solving a *PoW* puzzle: the *coinbase* pays the solver (as in b-money), here Alice



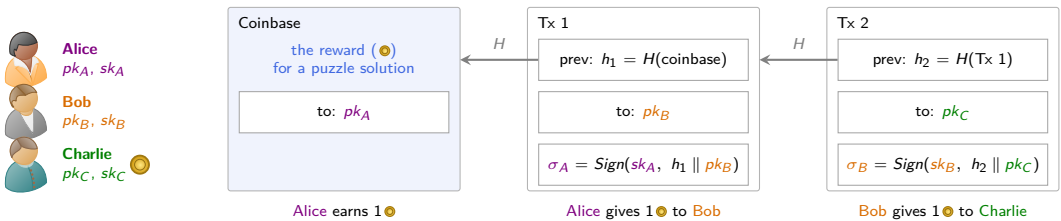


Alice earns 1

- a coin is **minted** by solving a *PoW* puzzle: the *coinbase* pays the solver (as in b-money), here Alice
- its **ownership** is a chain of transactions: the owner signs, with its secret key, the hash of the previous transaction and the public key of the next owner



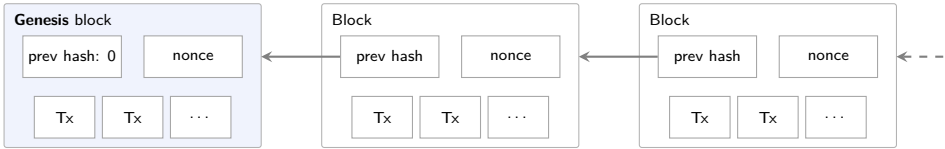
- a coin is **minted** by solving a *PoW* puzzle: the *coinbase* pays the solver (as in b-money), here Alice
- its **ownership** is a chain of transactions: the owner signs, with its secret key, the hash of the previous transaction and the public key of the next owner
- the payee, Charlie, checks each signature with the public key named in the previous transaction, back to the coinbase



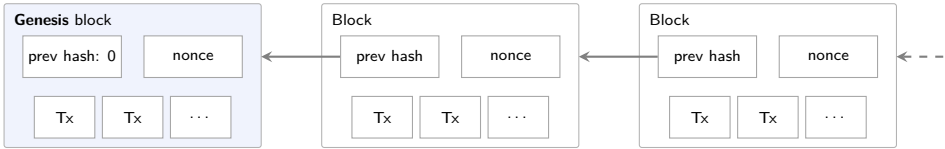
But how to verify that there is **no double spending**?

- with a bank: a central server timestamps blocks of transactions and defines a global order; the first spend wins
- how to implement timestamping in a **permissionless** setting, where anyone can join and leave (churn)?

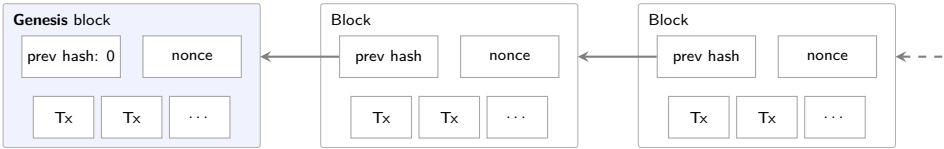
Use again PoW!



1. each process maintains a locally consistent copy of the blockchain, which starts with a fixed first block, the **genesis** block

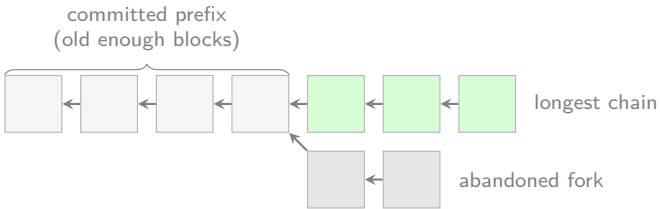


1. each process maintains a locally consistent copy of the blockchain, which starts with a fixed first block, the **genesis** block
2. to update it, a process solves a time-consuming puzzle (PoW). To append (*mine*) a block B with transactions T, it broadcasts $B = \langle h, T, n \rangle$ such that
   - h is the hash of the previous block
   - n is the nonce **to be found by the miner**
   - $H(h \,\|\, T \,\|\, n)$ contains δ zeros (in practice, is below a target)
   - T is committed in the header through its Merkle root



1. each process maintains a locally consistent copy of the blockchain, which starts with a fixed first block, the **genesis** block
2. to update it, a process solves a time-consuming puzzle (PoW). To append (*mine*) a block B with transactions T, it broadcasts $B = \langle h, T, n \rangle$ such that
   - h is the hash of the previous block
   - n is the nonce **to be found by the miner**
   - $H(h \,\|\, T \,\|\, n)$ contains δ zeros (in practice, is below a target)
   - T is committed in the header through its Merkle root
3. periodically, processes compare blockchains: a process continues with the **longest** verifiable one (in fact, the one with the most accumulated work)

Nakamoto consensus: forks and finality

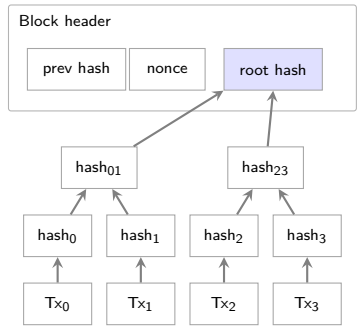


- akin to *eventual consistency* (e.g., Bayou): forks are temporary
- old enough blocks are considered committed: Bitcoin adds a block every 10 minutes and traces back 6 blocks (about an hour)
- the probability of a revert drops **exponentially** with the number of following blocks
- the difficulty δ is adjusted every 2016 blocks to keep one block per 10 minutes

- miners are rewarded with newly minted coins (the *coinbase*, halved every 210,000 blocks) and with fees
  - the **incentive** to follow the protocol
  - the answer to b-money's minting question
- each transaction is a simple script (not Turing-complete, no loop); richer programs come with Ethereum

Comparing chains is **very expensive**. To solve this, Bitcoin uses Merkle trees, instead of the transactions themselves.

- leaves = transactions
- internal nodes = hashes of children
- the header commits to all transactions through the root hash



Key assumption

The adversary does not control more than **half** of the computing power.

Key assumption

The adversary does not control more than **half** of the computing power.

Pros

- permissionless
- resists Sybil attacks
- simple, and robust for 15+ years

Cons

- slow throughput (a few txs/s)
- high latency (termination w.h.p.)
- very energy hungry

Key assumption

The adversary does not control more than **half** of the computing power.

Pros

- permissionless
- resists Sybil attacks
- simple, and robust for 15+ years

Cons

- slow throughput (a few txs/s)
- high latency (termination w.h.p.)
- very energy hungry

Other PoW chains: Litecoin, Monero; Ethereum used PoW until September 2022.

Next: classical consensus (Paxos, PBFT; lectures 3–5) and Proof-of-X (lecture 6).

Outline
5. Wrap-up

1. Context: why decentralize?
2. Blockchain 101
3. Safe computing
4. Bitcoin: the first blockchain
5. Wrap-up

Takeaways

- the Internet is decentralized in its protocols but centralized in its services; blockchains try to decentralize *trust* itself
- blockchains combine two lineages: distributed computing (networks, p2p, replication; lectures 2–5) and safe computing (signatures, hashes, proof-of-work)
- **Merkle trees**: a short commitment and a logarithmic proof of membership; the base of block headers and light clients
- **proof-of-work** turns scarce computing power into a Sybil-resistant identity; Nakamoto consensus is eventually consistent and probabilistic
- Bitcoin assumes less than half of the computing power is adversarial; its cost: low throughput, high latency, energy
- blockchains serve payments, smart contracts, tokens, DAOs, and DeFi (lectures 7–9); the rest of the course studies how they work, and how to do better

- **practical 1**: Merkle trees
- **lecture 2**: data dissemination and mempool (p2p, flooding, gossip)

Readings

- the Bitcoin white paper [†]
- Dwork and Naor, *Pricing via processing or combatting junk mail* [‡]
- Back, *Hashcash: a denial of service counter-measure* [§]

[†] S. Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. Cryptography mailing list. 2008.

[‡] C. Dwork et al. "Pricing via Processing or Combatting Junk Mail". *CRYPTO*. 1992.

[§] A. Back. *Hashcash — A Denial of Service Counter-Measure*. Technical report. 2002.

- answer the quiz of lecture 1, from your phone or your computer
- scan the QR code, or follow the link below



https://digistorm.app/p/9561847